



HIDDEN WUMPUS WORLD GAME

AI Project

Instructor Name: Sir Shahid Ali

Course Code: Artificial Intelligence CSC-

Submitted by:

Name

Registration No.

Menahil Suleman

FA24-BDS-029

Date: 28 - 11 - 2025

Institute: COMSATS University Islamabad

Contents

.....	0
Abstract.....	3
Introduction.....	4
Objectives.....	4
1. Develop a Grid-Based Adventure Environment	4
2. Implement Player Mechanics.....	4
3. Introduce Hazard Awareness and Percepts	4
4. Integrate AI Agent	4
5. Build an Interactive GUI.....	5
6. Promote Strategic Gameplay	5
Literature Background.....	5
Modules.....	6
User Interface Overview.....	6
Main Menu	6
.....	7
In-Game Interface (HUD + Controls)	7
On-Screen Indicators.....	8
Methodology / Project Implementation	8
Algorithms Used.....	8
1. Grid and Environment Setup.....	9
2. Player Mechanics	10
3. Item Collection	11
4. Arrow Shooting & Wumpus Interaction	11
5. Perception System (Breeze & Stench).....	11
6. AI Integration & Safe Exploration.....	11
7. Graphical Rendering (Tkinter GUI)	12
8. Game Logic & State Management	13
Limitations.....	13
Results & Discussion.....	13
Manual Gameplay.....	14
AI Agent Gameplay	14
Conclusion & Future Work	15

Conclusion	15
Future Work.....	15

Abstract

This project presents a Python-based implementation of the classic Wumpus World game, a grid-based environment commonly used in artificial intelligence studies to demonstrate decision-making and reasoning under uncertainty. The game simulates a hazardous world containing pits, a Wumpus, and hidden gold, where the player must navigate safely using percepts such as breezes and stench to avoid dangers and reach the goal. The project emphasizes environment representation, player interaction, and logical reasoning, while providing a turn-based gameplay system that tracks actions, hazards, and scores. Through this project, concepts of AI, problem-solving, and programming in Python are explored, demonstrating practical applications of computational logic and strategic planning in uncertain environments.

Introduction

The **Hidden Wumpus World Game** is a grid-based adventure inspired by the classic AI problem “Wumpus World.” The player navigates 8x8 grid filled with hidden hazards like pits and a Wumpus, while collecting valuable items such as gold and arrows. Players must rely on clues from adjacent tiles, a breeze near pits and a stench near the Wumpus—to make safe decisions.

The game combines strategy, decision-making, and resource management, allowing players to move, grab items, and shoot arrows to defeat the Wumpus. It demonstrates key AI concepts, including exploration, perception, and planning, in an interactive environment built using Python Tkinter.

- Navigate safely to collect gold and avoid hazards.
- Use arrows strategically to kill the Wumpus.
- Receive perceptual hints (breeze, stench) from adjacent tiles.
- Score is based on collecting items, killing the Wumpus, and avoiding penalties.

Objectives

1. Develop a Grid-Based Adventure Environment

- Create a structured grid where the player navigates safely.
- Implement random placement of hazards (pits, Wumpus) and collectibles (gold, arrows).

2. Implement Player Mechanics

- Enable movement in four directions using arrow keys.
- Allow interaction with items: grabbing gold/arrows and shooting arrows to kill the Wumpus.
- Maintain a scoring system to reward skillful actions and penalize mistakes.

3. Introduce Hazard Awareness and Percepts

- Provide visual and textual cues (stench for Wumpus, breeze for pits) to aid decision-making.
- Ensure player can infer safe paths using percepts.

4. Integrate AI Agent

- Support an agent capable of safe exploration using A* pathfinding.



FIGURE 1 GRID

Artificial Intelligence Ai

- Track safe tiles, detect hazards, and plan movements automatically.

5. Build an Interactive GUI

- Render all game elements visually using Tkinter.
- Update the HUD in real-time to display score, remaining arrows, and percept alerts.

6. Promote Strategic Gameplay

- Encourage players to plan movements carefully to survive hazards.
- Reward efficient completion: collecting gold and/or killing the Wumpus.

Literature Background

The concept of Wumpus World originates from classical artificial intelligence (AI) research, specifically in the study of knowledge representation, reasoning, and agent-based decision making. It was first introduced in the context of AI to illustrate how an agent can operate in a partially observable environment with uncertainty and hidden hazards.

In the original Wumpus World model:

- The agent explores a grid-based cave containing pits, gold, and the Wumpus, a monster that can kill the agent.
- The agent cannot see the entire environment at once, only **percepts** from adjacent tiles, such as **stench** near the Wumpus or **breeze** near pits.
- The agent must **deduce safe paths** and decide actions such as moving, grabbing gold, or shooting an arrow to kill the Wumpus.

My Project adaptations of Wumpus World, integrate:

- Interactive gameplay with a graphical user interface (GUI) using Python's Tkinter library.
- Multiple items, including gold for rewards and arrows for hunting the Wumpus.
- Real-time perceptual feedback to guide the player's decisions.
- Scoring mechanisms to encourage strategic planning rather than random movement.

This project combines the classical AI Wumpus World model with modern programming techniques, providing a hands-on demonstration of AI principles such as perception, planning, and reasoning under uncertainty.

Modules

Module	Description
World Generation	Initializes the grid, places the Wumpus, pits, gold, and arrows, ensuring valid positions.
Player Actions	Handles movement, grabbing items, shooting arrows, and score updates.
Arrow & Wumpus Interaction	Implements arrow shooting mechanics and Wumpus death logic.
Percept System	Provides feedback (stench, breeze) based on hazards in adjacent tiles.
AI Agent	Implements A* pathfinding, safe tile detection, and AI-driven decision-making.
GUI Rendering	Uses Tkinter to display the grid, player, items, agent path, and game state updates.
HUD & Scoring	Updates the score, arrows, and status messages in real-time.
Game Logic & Events	Maintains game state, checks for collisions, hazards, victory, or defeat conditions.

User Interface Overview

Main Menu

The game begins with a simple and interactive main menu offering the following options:

Menu Option	Description
New Game	Starts a fresh game by generating a new grid, hazards, gold, and player position.
About	Displays game rules, mechanics, and project information.
Quit	Exits the application.

MAIN MENU:



FIGURE 2 MAIN MENU

In-Game Interface (HUD + Controls)

During the game, a top control bar provides essential options and real-time information:

HUD Element	Purpose
Restart	Regenerates the world and restarts the game instantly.
Quit	Exits to the main menu.
AI Agent	Lets the AI explore, collect gold, and return safely using A*.
Arrows Counter	Shows remaining arrows.
Gold Counter	Shows collected gold.
Inventory	Displays items currently held by the player.
Info Bar	Shows percepts (stench, breeze), warnings, and status messages.

HUD:



FIGURE 3 HUD

INVENTORY:



FIGURE 4 INVENTORY BEFORE

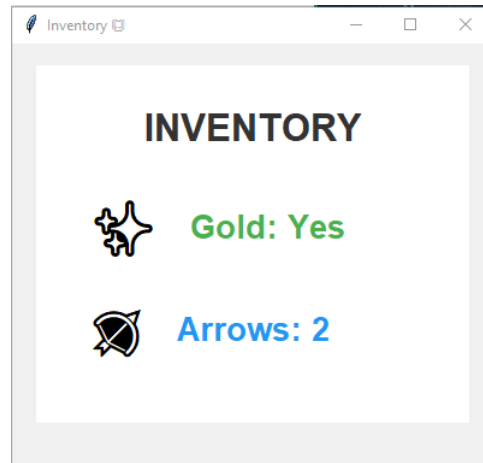


FIGURE 5 INVENTORY AFTER

On-Screen Indicators

Indicator	Meaning
Breeze	Pit nearby.
Stench	Wumpus nearby.
Safe Tile Highlight	Tile confirmed safe by the AI.
Agent Path (Lines)	Shows the route calculated using A*.

Methodology / Project Implementation

The **Hidden Wumpus World Game** is developed in **Python** using the **Tkinter** library to render a fully interactive grid-based environment. The game combines GUI programming, logical reasoning, and agent-based exploration. This section explains how the world is generated, how the player and AI interact with the environment, and how game logic is maintained throughout gameplay.

Algorithms Used

Algorithm Type	Description
Local-Based Constraint Reasoning	Uses percepts (breeze/stench) to infer safe or dangerous neighboring tiles.
Probabilistic Inference	Updates probabilities for pits and Wumpus to handle uncertainty and risk.

Algorithm Type	Description
Knowledge-Based Reasoning	Maintains a knowledge base of visited tiles and percepts to support informed decisions.
Heuristic Decision Making	Chooses actions and moves based on expected risk, Manhattan distance, and probabilities.
A* Search Algorithm	Finds optimal paths from the current position to targets (gold, safe tiles, or home).
Risk-Based Path Selection	Computes expected loss for each tile and selects the safest exploration path.
Rule-Based Action Selection	Uses IF-ELSE logic to determine whether to move, shoot, grab, or retreat.

1. Grid and Environment Setup

- The game world is represented as a $\text{SIZE} \times \text{SIZE}$ grid (8×8).
- Each tile is a Python set, capable of storing multiple elements such as "pit", "wumpus", "gold", or "arrow".
- The `create_world()` method initializes the grid with randomized placement of items while ensuring:
 - The starting tile (0,0) is always safe.
 - No pits or Wumpus are placed adjacent to the start.
 - Major items never overlap.



FIGURE 6 GRID ENVIRONMENT

Items placed include:

- Wumpus
- Multiple pits
- Gold pieces
- Arrows
- Breeze
- Stench

This setup ensures each game instance is unique while still fair.

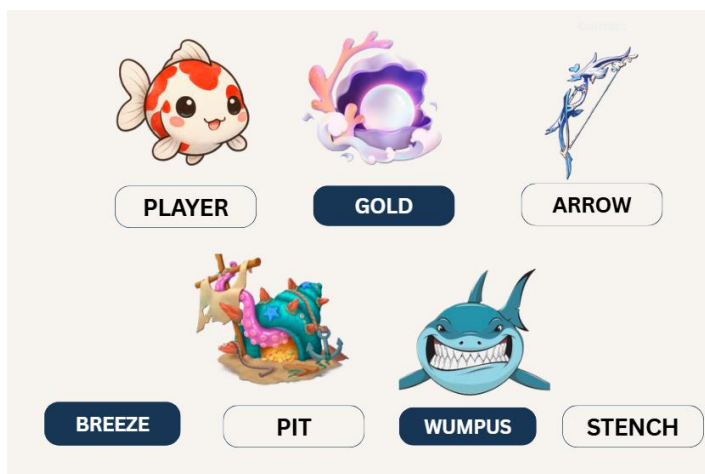


FIGURE 7 GAME ENTITIES

2. Player Mechanics

The player begins at **(0,0)** and interacts through keyboard controls:

- **Move:** Up, Down, Left, Right
- **Grab:** Collects gold or arrows on the current tile
- **Shoot:** Fire an arrow at an adjacent tile where wumpus is.

Additional rules:

- Each movement costs -1 score.
- Moving into a pit or live Wumpus results in instant death.
- Hitting a wall gives a -5 score penalty without movement.

The player must use clues, arrows, and cautious navigation to survive and win.

3. Item Collection

- **Gold:** Increases score and is removed from the tile after grabbing.
- **Arrows:** Increase ammo count, allowing the player to shoot again.
- Items automatically disappear after being collected.

4. Arrow Shooting & Wumpus Interaction

- Arrows can only be shot to tiles directly adjacent (up, down, left, right).
- Successfully hitting the Wumpus:
 - Gives +500 score
 - Marks the Wumpus as dead
 - Removes danger a
- Missing the shot or shooting without arrows results in a small penalty.

This mechanic introduces risk, timing, and strategy.

5. Perception System (Breeze & Stench)

To mimic classic AI logic, the player receives percepts from nearby tiles:

- **Stench:** Adjacent to the Wumpus
- **Breeze:** Adjacent to pits

These percepts appear in the HUD, helping players infer hidden hazards and plan safer routes.

6. AI Integration & Safe Exploration

The game includes an optional AI agent capable of navigating the world intelligently.

AI Features:

- **Safe Tile Identification:** Marks tiles proven safe based on percepts.
- **Pathfinding:** Implements A* algorithm to reach goals such as gold while avoiding dangers.
- **Knowledge Updating:** Adjusts internal map based on stench/breeze clues and Wumpus death.
- **Visual Highlights:** Shows safe tiles and the agent's path on the grid.

This AI support demonstrates reasoning and decision-making under uncertainty.

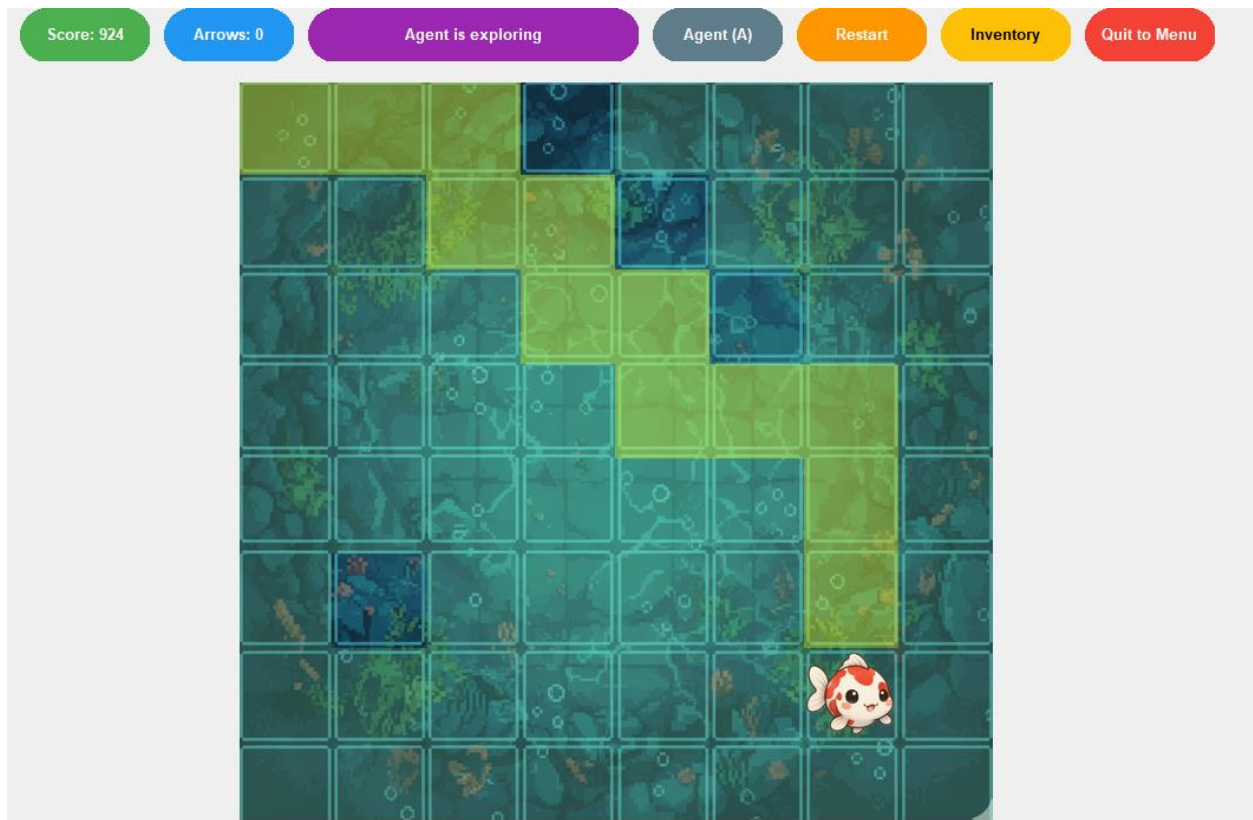


FIGURE 8 AI AGENT

7. Graphical Rendering (Tkinter GUI)

The entire game is displayed using a Tkinter canvas.
Visual elements include:

- Player sprite
- Wumpus (alive/dead)
- Pits
- Gold
- Arrows
- Safe tile highlights
- AI-generated paths

The GUI updates dynamically after every action — movement, grabbing, shooting, or agent decisions.

Along with the grid, a HUD panel displays:

- Score
- Remaining arrows
- Percepts (breeze, stench)

- Status messages

8. Game Logic & State Management

The game maintains key internal states such as:

- **player_pos**
- **wumpus_alive**
- **score**
- **game_over**

The game loop for each action works as follows:

1. Player performs action (move / grab / shoot)
2. System checks for hazards (pits, Wumpus)
3. Score and percepts update
4. GUI is re-rendered
5. Game ends if player dies or completes objectives

Win conditions:

- Player collects gold and/or kills the Wumpus
- Player survives without stepping into hazards

Limitations

Limitation	Description
Fixed Grid Size	Only 8×8 world; limits complexity.
Random Placement	Some worlds may become too easy or too hard.
Basic AI Logic	AI uses simple reasoning, not advanced inference.
Different Win Conditions	Manual mode: win by killing Wumpus. AI mode: win by grabbing gold + returning to start.
Simple Graphics	Tkinter limits animations and visual quality.
No Dynamic World	Environment does not change after generation.
Fixed Scoring	Users cannot modify scoring rules.
No Sound/UI Effects	Minimal interface due to library limitations.

Results & Discussion

The Hidden Wumpus World game demonstrates how an agent or player can navigate a hazardous environment while collecting items and avoiding dangers. The results vary depending on whether the game is played manually by a user or automatically by the AI agent.

Manual Gameplay

- Players control movement using arrow keys and can shoot arrows with Shift + Arrow keys.
- The player collects gold and arrows, avoids pits, and may encounter the Wumpus.
- Scoring reflects skillful play: collecting gold gives +1000 points, arrows give +10, and killing the Wumpus gives +500.
- Game outcomes:
 - **Winning:** Collecting gold and killing the Wumpus (optional).
 - **Losing:** Falling into a pit or being eaten by a live Wumpus.

AI Agent Gameplay

- The AI explores the grid using A* pathfinding and percepts (stench for Wumpus, breeze for pits).
- The AI automatically marks safe tiles and follows optimal paths to collect gold and reach the starting point.
- Advantages:
 - Reduces human error.
 - Ensures strategic gameplay and higher chance of winning.
- Outcomes:
 - **Winning:** AI collects gold and returns to the starting tile.
 - **Losing:** Only if the AI cannot find a safe path or all paths are blocked by hazards.

Gameplay Type	Collected Gold	Arrows Used	Wumpus Status	Score Outcome	Result
Manual	Yes/No	Varies	Alive/Dead	Varies	Win/Loss
AI Agent	Yes	Optimized	Alive/Dead	Optimized	Win/Loss

Key Observations:

- Manual gameplay depends heavily on player skill and can be unpredictable.
- AI gameplay ensures the agent safely collects items and maximizes score, often outperforming manual play.
- The percept system (stench/breeze) effectively guides decision-making, especially for AI.
- Arrow shooting provides a strategic tool to neutralize the Wumpus and improve winning chances.

Conclusion & Future Work

Conclusion

The Hidden Wumpus World game successfully demonstrates a grid-based AI environment where hazards, rewards, and player/agent interactions create strategic challenges. The game allows both manual and AI-driven gameplay:

- Manual play tests the player's decision-making, reflexes, and risk management.
- The AI agent leverages percepts, safe tile mapping, and A* pathfinding to optimize movements, collect gold, and avoid hazards.
- Arrow shooting introduces a tactical element for neutralizing the Wumpus, adding depth to gameplay.
- The percept system (stench for Wumpus, breeze for pits) effectively guides safe navigation.

Overall, the project combines classical AI principles with interactive gameplay, teaching hazard reasoning, strategic planning, and AI-driven decision-making.

Future Work

The game can be enhanced further with:

- **Dynamic Difficulty Levels:** Increase grid size, add more hazards, or multiple Wumpuses for advanced challenges.
- **Improved AI Intelligence:** Incorporate reinforcement learning for adaptive decision-making.
- **Multiplayer Mode:** Enable cooperative or competitive gameplay between multiple agents or players.
- **Extended Item System:** Introduce new items (like traps, shields, or extra points) to diversify strategy.
- **Better Graphics & Animations:** Upgrade visuals for a more engaging user experience.
- **Scoring Analytics:** Track performance over multiple games to analyze strategies and improvement.

This ensures the Hidden Wumpus World game remains both educational and entertaining while offering potential for advanced AI experimentation.